

Caso Práctico 2: MediCare - Gestión de Citas Médicas y Usuarios

Valor: 15% de la nota final del curso

Indicaciones y Normas de la Prueba

- Siguiendo la directriz de honestidad académica, cualquier intento de fraude durante la solución del ejercicio práctico autoriza al docente a la anulación de esta actividad evaluativa, consignando una nota de cero.
- Puede utilizar TODAS las filminas del curso y proyectos desarrollados previa o simultáneamente en clase.
- NO se permite que en su respuesta aparezcan nombres de variables exactamente iguales a los proyectos de clase (evitar copiar y pegar literal).
- Escriba respetando el formato adecuado, convenciones de nombres de clases, variables y métodos (Java/CamelCase). Se puede perder hasta un 30% por incumplir este aspecto.
- La sesión será grabada y los estudiantes deben activar la cámara para la realización de la prueba.
- El micrófono deberá permanecer apagado, a menos que el estudiante tenga una consulta.

Descripción del Ejercicio Práctico

Desarrolle una aplicación web para la plataforma de servicios de salud 'MediCare' que permita administrar los usuarios del sistema, gestionar sus roles, restringir el acceso a funcionalidades según permisos en Spring Security y consultar información relevante sobre las Citas Médicas del centro de salud. Además, los usuarios podrán registrarse y recibir un correo electrónico automático de confirmación/bienvenida utilizando Spring Mail.

1. Repositorio en GitHub

Cree un repositorio en GitHub llamado con el siguiente formato exacto: **EjercicioPractico2_TUNOMBRE**

2. Configuración e Inicialización del Proyecto

Utilice Spring Boot Initializr (Java + Spring Boot) con las siguientes dependencias obligatorias:

- Spring Web
- Spring Data JPA
- MySQL Driver
- Spring Mail

- Spring Security
- Thymeleaf

Estructura requerida de paquetes: domain, repository, service, controllers, config, templates.

Configuración de Puerto: La aplicación debe ejecutarse en el puerto **78** (application.properties: server.port=78).

3. Conexión y Script de Base de Datos

Configure la conexión a MySQL utilizando la base de datos 'medicare' según el siguiente script oficial:

```
DROP DATABASE IF EXISTS medicare;
CREATE DATABASE medicare
  CHARACTER SET = utf8mb4
  COLLATE = utf8mb4_unicode_ci;
USE medicare;

-- Tabla roles
CREATE TABLE rol (
  id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  nombre VARCHAR(100) NOT NULL UNIQUE
);

-- Tabla usuarios
CREATE TABLE usuario (
  id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  nombre VARCHAR(150),
  email VARCHAR(200) UNIQUE,
  password VARCHAR(255),
  rol_id BIGINT UNSIGNED,
  activo BOOLEAN DEFAULT TRUE,
  fecha_creacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (rol_id) REFERENCES rol(id)
);

-- Tabla citas medicas
CREATE TABLE cita_medica (
  id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  paciente_nombre VARCHAR(150),
  especialidad VARCHAR(100),
  fecha DATE,
  costo DOUBLE,
  activa BOOLEAN DEFAULT TRUE
);

-- Datos de prueba
INSERT INTO rol (nombre) VALUES ('ADMIN'), ('MEDICO'), ('PACIENTE');

INSERT INTO usuario (nombre, email, password, rol_id) VALUES
('Admin General', 'admin@medicare.com', '12345', 1),
('Dr. Roberto', 'medico@medicare.com', '12345', 2),
('Paciente Maria', 'paciente@medicare.com', '12345', 3);

INSERT INTO cita_medica (paciente_nombre, especialidad, fecha, costo, activa) VALUES
```

```
('Carlos Gómez', 'Cardiología', '2026-05-10', 45000.0),  
('Ana Martínez', 'Dermatología', '2026-06-15', 35000.0);
```

4. Funcionalidades Requeridas

A. Gestión de Usuarios (CRUD Completo)

Permite listar, crear, editar, eliminar y ver detalle del usuario.

Cada usuario debe estar asociado a un rol.

Al crear o registrar un usuario, se debe enviar automáticamente un correo electrónico de bienvenida utilizando Spring Mail.

B. Gestión de Roles

CRUD completo para la administración de roles.

Cada usuario posee asignado un rol específico.

Roles requeridos en el sistema: ADMIN, MEDICO, PACIENTE.

C. Gestión de Citas Médicas

CRUD completo de Citas Médicas.

Atributos requeridos de la cita: Nombre del Paciente, Especialidad, Fecha de la Cita, Costo y Estado (activa/inactiva).

D. Control de Acceso con Spring Security

Módulo de autenticación con Login y Logout.

Restricción de acceso granular según el rol del usuario autenticado:

- ADMIN: Gestiona Usuarios, Roles y Citas Médicas.
- MEDICO: Gestiona Citas Médicas.
- PACIENTE: Visualiza Citas Médicas.

Redirección personalizada según rol tras iniciar sesión.

E. Consultas Avanzadas con JPA / Hibernate

Implementar al menos 3 consultas derivadas o personalizadas (@Query) en los repositorios, como por ejemplo:

- Buscar citas médicas por estado (activas/inactivas).
- Buscar citas médicas dentro de un rango de fechas.
- Buscar citas por coincidencia parcial en el nombre de la especialidad.
- Buscar usuarios por rol asignado o contar citas activas.

Crear una vista dedicada en la interfaz para mostrar los resultados de las consultas.

F. Interfaz Gráfica con Thymeleaf y Bootstrap

Uso de plantilla base (layout) reutilizable con Header, Navbar y Footer.

Páginas mínimas requeridas: Página Principal, Gestión de Usuarios, Gestión de Roles, Gestión de Citas Médicas, Login y Página de Consultas Avanzadas.

5. Rúbrica de Evaluación

La evaluación se realizará estrictamente conforme al cumplimiento de los siguientes aspectos:

Criterio de Evaluación	Descripción	Puntos Máx.	Cumple (100%)	Parcial (50%)	No Cumple (0%)
Creación del Repositorio	El repositorio en GitHub está correctamente creado y nombrado como EjercicioPractico2_TUNOMBRE con el proyecto completo.	5	5	2.5	0
Conexión a BD	Configuración correcta y funcional de la conexión a MySQL usando la base de datos 'medicare'.	5	5	2.5	0
Organización de Paquetes	Paquetes domain, repository, service, serviceimpl, controllers y config organizados correctamente.	15	15	7.5	0
Lógica en Repositorios (DAO)	Repositorios JPA bien estructurados con las consultas derivadas/personalizadas para Citas Médicas y Usuarios.	10	10	5	0
Lógica en Paquete Service	Interfaces Service y sus implementaciones con la lógica de negocio y la integración de Spring Mail.	15	15	7.5	0
Controladores (Controllers)	Manejo adecuado de peticiones GET y POST, redirecciones por rol y flujo de datos backend-frontend.	15	15	7.5	0
Creación de Vistas	Vistas HTML estructuradas dinámicamente con Thymeleaf para Usuarios, Roles, Citas y Consultas.	20	20	10	0
Funcionalidad General y Security	Cumplimiento del CRUD completo, Login/Logout, permisos por rol (ADMIN, MEDICO, PACIENTE) y envío de correos.	10	10	5	0
Estética e Interfaz	Interfaz clara, ordenada y visualmente atractiva con Bootstrap, plantilla base (Header, Navbar, Footer) y usabilidad.	5	5	2.5	0
TOTAL	Puntuación Máxima Obtenible	100 pts			